

ShikshaSetu AI

Complete Project Workflow & First-Round Implementation Plan

Goal: Build a personalized, multilingual AI learning system that connects a student's specific confusion to the right explanation, at the right level and in the right language.

1. Final Product Scope

Core product: ShikshaSetu AI — a personalized multilingual learning companion with two interfaces: Student and Teacher.

Student	Teacher
Ask doubts via text/image (voice optional)	Class overview
Grounded step-by-step explanation	Students needing attention
Preferred language adaptation	Concept-level mastery
Trusted source citations	Risk reasons
Diagnostic questions	AI intervention recommendations
Adaptive practice	Student detail view
Concept mastery/progress	Learning trend

2. Final Technology Stack — LOCK THIS

Layer	Technology	Purpose
Frontend	Next.js + TypeScript	Student/teacher web application
UI	Tailwind CSS + shadcn/ui	Fast, polished responsive UI
Charts	Recharts	Mastery and teacher analytics
Backend	Python + FastAPI	APIs and AI orchestration
LLM	Gemini API	Explanation, concept detection, practice, insights
Embeddings	Gemini Embeddings	Semantic retrieval
Database	PostgreSQL	Users, concepts, attempts, mastery, sources
Vector search	pgvector	Educational-content retrieval
Deployment	Vercel + Render + PostgreSQL provider	Live frontend, backend and database
Version control	GitHub	Team collaboration and submission

3. Recommended Knowledge Base

For the first round, keep the knowledge base intentionally small and reliable. Focus on **Class 8 Science** and approximately 2–4 chapters. Use open/public educational material with clear source attribution. DIKSHA is a strong India-focused candidate because it provides curriculum-oriented resources and supports multilingual learning.

Do not attempt Class 1–12, every subject and every Indian language before the first-round deadline. A narrow, reliable knowledge base is better than a broad, unreliable one.

4. Complete System Architecture

Student / Teacher → Next.js → FastAPI → AI Agents + Learning Engine → RAG → PostgreSQL/pgvector → Gemini → Response → Student/Teacher dashboards

Core components: Student Portal, Teacher Portal, FastAPI Backend, Doubt Agent, Practice Agent, Teacher Insight Agent, RAG Engine, Student Mastery Engine, PostgreSQL and pgvector, Gemini API.

5. Complete Student Workflow — End to End

Step 1 — Student Profile

Student selects grade, subject and preferred language. Example: Class 8, Science, Assamese.

Step 2 — Ask a Doubt

Student submits a text question or image. Voice can be added if time permits. All modalities are converted into a text representation for the main AI pipeline.

Step 3 — Concept Detection

Gemini identifies subject, concept, sub-concept and approximate difficulty. Example: Science → Electric Circuit → Open Circuit → Beginner.

Step 4 — Retrieve Learning Profile

Backend reads the student's concept-level mastery. Example: Electric Circuit 42%, Current 58%, Conductors 81%.

Step 5 — RAG Retrieval

The question is embedded and matched against trusted educational chunks stored in PostgreSQL/pgvector. Relevant chapter/page metadata is retrieved with the content.

Step 6 — Personalized Explanation

Gemini receives the student level, language, mastery context and retrieved evidence. It generates a simple, step-by-step explanation grounded in the source.

Step 7 — Citation

The UI shows the educational source, chapter and page/section so the student can distinguish source material from AI explanation.

Step 8 — Diagnostic Question

The AI asks a small concept-check question. The student's answer provides evidence about what they actually understand.

Step 9 — Misconception Detection

The system evaluates the answer and identifies a likely misconception rather than only marking it correct/incorrect.

Step 10 — Targeted Intervention

If incorrect, the tutor gives a hint or simpler explanation and asks the student to retry.

Step 11 — Adaptive Practice

The system generates 3 targeted questions around the weak concept, with difficulty adjusted to the student's current level.

Step 12 — Record Attempt

Every attempt stores student, question, concept, correctness, time and optionally hint usage.

Step 13 — Update Mastery

The learning engine updates the student's concept mastery score using recent performance and repeated mistakes.

Step 14 — Student Progress

The dashboard shows improvement, weak concepts and recommended next practice.

6. Example Student Journey

Question: “Why does a bulb not glow when the switch is open?”

Detected concept: Electric Circuit → Open Circuit

Mastery: 42%

RAG: Retrieves trusted Science textbook content

Response: Simple explanation in Assamese + source citation

Diagnostic: Student answers incorrectly

Misconception: Student thinks the battery stops working

Practice: 3 targeted circuit questions

Result: 2/3 correct

Mastery: 42% → 56%

Teacher: Student is now improving instead of remaining at risk.

7. RAG Workflow

Educational PDF/source → text extraction → chapter/section detection → chunking → metadata → embeddings → pgvector → semantic retrieval → optional reranking → grounded Gemini prompt → explanation + citations.

Data	Metadata to preserve
Text chunk	subject, grade, chapter, topic
Source	title, publisher/source, URL
Location	page/section
Language	language code/name

8. Personalization / Mastery Engine

Track mastery per concept instead of only total quiz score. A simple interpretable score is sufficient for the first round.

Example: correct answer +0.10; incorrect -0.08; repeated mistake -0.12; correct after hint +0.04. Clamp the score to 0-1.

Interpretation: 0-0.30 = needs foundation; 0.30-0.60 = developing; 0.60-0.80 = good; 0.80-1.00 = mastered.

9. Teacher Insight Workflow

Student attempts → concept mastery → repeated-error analysis → risk score → teacher dashboard → AI explanation of why the student is flagged → recommended intervention.

Example: Student has 5 recent errors in fractions, mastery 29%, and repeatedly adds denominators directly. Teacher sees: “High risk — repeated denominator misconception. Recommended intervention: visual fraction model + 3 foundational questions.”

10. Suggested Database Tables

Table	Important fields
users	id, name, email, role
students	user_id, grade, preferred_language
concepts	id, subject, name, grade, difficulty
sources	id, title, chapter, page, URL, language
questions	id, concept_id, difficulty, question, answer, source_id
attempts	student_id, question_id, concept_id, correct, time_taken
student_mastery	student_id, concept_id, mastery_score, attempts, last_attempt

11. API Plan

Endpoint	Purpose
POST /api/doubt	Analyze doubt, retrieve evidence and return personalized answer
POST /api/practice/generate	Generate targeted practice for a weak concept
POST /api/practice/submit	Evaluate attempt and update mastery
GET /api/student/progress	Return mastery and learning progress
GET /api/student/mastery	Return concept-level mastery
GET /api/teacher/dashboard	Class metrics and at-risk students
GET /api/teacher/student/{id}	Detailed student insight

12. Three-Person Team Responsibilities

Person	Ownership	Deliverables
Person 1 — Frontend/UX	Next.js, Tailwind, shadcn, UI	Landing, student flow, practice, progress, teacher dashboard, responsive polish
Person 2 — Backend/DB	FastAPI, PostgreSQL, APIs	Schema, endpoints, CRUD, attempts, mastery storage, integration
Person 3 — AI/RAG	Gemini, RAG, learning intelligence	Ingestion, embeddings, retrieval, prompts, concept detection, misconception, practice, teacher insights

13. GitHub Structure

```
shiksha-setu/  
■■■ frontend/  
■■■ backend/  
■■■ data/  
■■■ docs/  
■■■ README.md
```

Use one repository. Each person can work on a separate branch and merge into main. Avoid separate frontend/backend/AI repositories unless integration becomes unavoidable.

14. Five-Day Execution Plan

Date	Main objective	Required result
Aug 18	Foundation	Next.js, FastAPI, DB, Gemini, initial RAG prototype
Aug 19	Grounded doubt solving	Question → RAG → personalized explanation → citation
Aug 20	Adaptive learning	Diagnostic → misconception → practice → mastery update
Aug 21	Teacher intelligence	Dashboard → risk → reason → intervention recommendation
Aug 22	Freeze + deploy	Bug fixing, responsive polish, live deployment, README, final test
Aug 23	Submission	Final testing, demo video, GitHub link, live link, submission

15. MVP Priority — What Must and Must Not Be Built

Priority	Feature
MUST	Student profile
MUST	RAG-based doubt solving
MUST	Trusted source citations
MUST	Multilingual response
MUST	Adaptive practice
MUST	Concept mastery
MUST	Teacher dashboard
MUST	Risk detection + reason
MUST	AI intervention recommendation
OPTIONAL	Image-based doubts
OPTIONAL	Voice input/output
SKIP	Scholarship matcher
SKIP	Parent dashboard
SKIP	Complex authentication
SKIP	Offline mode

16. Demo Video Workflow

Target length: 2.5–4 minutes.

- 0:00–0:20: Problem — student has a concept-level confusion.
- 0:20–0:40: Introduce ShikshaSetu.
- 0:40–1:30: Student asks doubt → multilingual grounded explanation → citation.
- 1:30–2:15: Diagnostic → misconception → adaptive practice → mastery improvement.
- 2:15–3:00: Teacher dashboard → at-risk student → reason → intervention recommendation.
- 3:00–3:40: Brief architecture + impact statement.

17. The Core Differentiator

Do not pitch it as “an AI chatbot for education.”

Pitch it as: “We don’t personalize the answer; we personalize the learning path.”

Normal chatbot: Question → Answer
ShikshaSetu: Question → Concept → Student knowledge state → Trusted evidence → Personalized explanation → Misconception → Targeted practice → New evidence → Updated knowledge state → Next learning action

18. Final End-to-End Workflow

STUDENT → ASK DOUBT → CONCEPT DETECTION → STUDENT MASTERY LOOKUP → RAG RETRIEVAL → TRUSTED SOURCE → GEMINI PERSONALIZED EXPLANATION → CITATION → DIAGNOSTIC QUESTION → MISCONCEPTION DETECTION → ADAPTIVE PRACTICE → ATTEMPT → MASTERY UPDATE → STUDENT PROGRESS + TEACHER ALERT → AI INTERVENTION

19. Team Rule for the Deadline

August 20 evening is the critical checkpoint: the complete student learning loop must work end-to-end. August 21 is for teacher intelligence. August 22 is feature freeze, deployment and polish. August 23 is submission only — no new features.

The team should work in parallel. Person 1 can use mock JSON while Persons 2 and 3 build APIs and AI. Integrate continuously rather than waiting until the final day.